

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Anti-Tamper Smart Parcel System

BECE320E- Embedded C
Course Project

Bachelor of Technology in Electronics and Communication

By

23BEC0087 - DONA MAXIN
23BEC0320 - SUDARSAN P J

14th April,2026

Declaration

We hereby declare that the project report entitled "Anti-Tamper Smart Parcel System" submitted in partial fulfilment of the requirements for the Embedded C Programming course is our own original work. The information and data used in this report have been collected and processed by us and have not been submitted elsewhere for any academic or professional purpose.

We further declare that all the sources consulted and referenced in this report have been duly acknowledged.

Dona Maxin — 23BEC0087 **Sudarsan P J — 23BEC0320**

Date: 14-04-2026

Abstract

The Anti-Tamper Smart Parcel System is an embedded security solution designed to detect and report unauthorized access to parcels during transit and storage. The system is built on the ESP32 microcontroller—featuring a dual-core Xtensa LX6 processor and integrated Wi-Fi—and employs a multi-sensor architecture for comprehensive, real-time tamper detection.

Three complementary sensors form the detection core: the MPU6050 six-axis accelerometer/gyroscope detects physical shocks, drops, and motion events; a Light Dependent Resistor (LDR) monitors for sudden light intrusion indicating box opening; and a DHT11 or DS18B20 temperature sensor flags heat-based tampering or adverse storage conditions. A hierarchical decision logic evaluates sensor readings against calibrated thresholds and triggers immediate alerts—via buzzer or LED—whenever a tamper condition is identified.

The system classifies parcel security into four states: SECURE, SHOCK/DROP, BOX OPENED, and HEAT TAMPER and results displayed on an OLED. All sensor events are logged continuously at 115200 baud over a serial monitor, enabling real-time debugging and audit trails. Programmed entirely in Embedded C on the Arduino/ESP32 framework, the project demonstrates practical skills in GPIO interfacing, I2C communication, ADC sampling, interrupt handling, and threshold-based decision logic.

This project is applicable in e-commerce delivery, pharmaceutical logistics, sensitive document transport, and laboratory sample security. Future enhancements include cloud connectivity for remote monitoring, tracking, and cryptographic tamper-evidence logging.

Table of Contents

S.No	Contents	Page No.
1.	Declaration	2
2.	Abstract	3
3.	Table of Contents	4
4.	List if Figures	4
5.	List of tables	5
6.	Chapter 1: Introduction	6-7
7.	Chapter 2: Literature Survey	8
8.	Chapter 3: Problem Statement	9
9.	Chapter 4: Methodology	10-11
10.	Chapter 5: System Design	12-13
11.	Chapter 6: Implementation	14
12.	Chapter 7: Results and Discussion	15-16
13.	Chapter 8: Testing & Validation	17
14.	Chapter 9: Cost Analysis	18
15.	Chapter 10: Conclusion	19
16.	Chapter 11: Future Work	20
17.	References	21

List of Figures

S.No	Contents	Page No.
1.	Fig(1)- Block Diagram	11
2.	Fig(2)- System Flow Chart	11
3.	Fig(3)- Architecture Diagram	12
4.	Fig(4)- Data flow diagram	13
5.	Fig(5)- Output Screenshot	16
6.	Fig(5)- Output Screenshot	16
7.	Fig(5)- Output Screenshot	16

List of Tables

S.No	Contents	Page No.
1.	2.2 Comparison of Approaches	8
2	5.2 Module-Level Design	12
3.	7.1 Sensor Output Observations	15
4.	8.1 Test Cases	17
5.	9.1 Hardware Cost	18

Chapter 1: Introduction

1.1 Background of the Topic

The rapid growth of e-commerce and global logistics has made secure parcel delivery a critical concern for businesses and end consumers alike. Traditional packaging and sealing methods offer limited resistance against tampering during transit, and detection of unauthorized access is often only possible post-delivery. The Internet of Things (IoT) combined with embedded sensor systems provides a powerful mechanism to monitor parcel integrity in real time.

Embedded systems based on low-power microcontrollers such as the ESP32 offer an ideal platform for compact, battery-operable, and wireless-capable tamper detection. By integrating multiple complementary sensors—accelerometers, light sensors, and temperature sensors—a robust multi-parameter tamper detection system can be realized at low cost and minimal power consumption.

1.2 Motivation / Need for the Project

Supply chain security lapses result in billions of dollars of losses annually due to theft, counterfeiting, and mishandling. High-value shipments—pharmaceuticals, electronics, legal documents—are particularly vulnerable. Existing solutions such as GPS trackers address location but not physical tampering. A low-cost, real-time, sensor-based tamper detection system embedded within the parcel itself addresses this gap directly.

This project was motivated by the desire to apply embedded C programming skills to a practical, impactful problem, demonstrating how a small microcontroller system can deliver meaningful security functionality with off-the-shelf components.

1.3 Objectives of the Project

- Design and implement a multi-sensor tamper detection system on the ESP32 platform.
- Detect three distinct tamper scenarios: physical shock/drop, box opening via light, and heat-based tampering.
- Implement real-time threshold-based decision logic in Embedded C.
- Provide immediate alert outputs through buzzer and/or LED indicators.
- Log all sensor events and tamper flags over a serial monitor for debugging and auditability.
- Demonstrate modular firmware design with separate sensor-reading, evaluation, and alert functions.

1.4 Scope and Limitations

The scope of this project covers the hardware integration, firmware development, and testing of the Anti-Tamper Smart Parcel System prototype. The system operates as a standalone embedded device without cloud connectivity in its current form. Limitations include:

- No wireless alerting (Wi-Fi/Bluetooth) implemented in the current prototype—alerts are local only.
- DHT11 has limited accuracy (±2°C) compared to more precise sensors.
- LDR threshold requires manual calibration based on ambient light conditions.
- No persistent data logging (EEPROM or SD card) in the current implementation.
- Battery life and power optimization have not been fully addressed.

Chapter 2: Literature Survey

2.1 Review of Existing Work

IoT-based parcel and asset monitoring has been an active area of research. An IEEE Access study on smart environmental monitoring [15] demonstrated that multi-sensor IoT nodes can achieve reliable real-time monitoring with minimal power. Wireless sensor networks for environmental monitoring [16] provided foundational approaches for sensor data aggregation and threshold alerting.

Fall detection and shock classification using accelerometers [17] established the use of three-axis accelerometers (as in the MPU6050) for reliable motion-event classification, with delta-g thresholds separating normal movement from drop events. Activity recognition surveys [18] confirmed the effectiveness of IMU-based detection in embedded low-power systems.

2.2 Comparison of Approaches

Approach	Sensors Used	Alert Method	Limitation
GPS-only tracker	GPS	SMS/App	No tamper detection
RFID seal	RFID	Reader scan	Passive, not realtime
Single accel. node	MPU6050 only	BLE/Wi-Fi	No light/temp layer
This Project	MPU6050+LDR+DHT11	Buzzer/LED/Serial	Local alert only

2.3 Research Gaps Identified

- Most existing systems rely on a single sensing modality; multi-parameter fusion for tamper detection is rare in low-cost embedded implementations.
- Light-based opening detection (LDR) is rarely combined with IMU-based shock detection in a unified parcel security node.
- Hierarchical tamper classification (priority-based decision trees) is underexplored in embedded C implementations for parcel security.

2.4 Justification for the Chosen Approach

The ESP32 with MPU6050, LDR, and DHT11 was selected for its cost-effectiveness, welldocumented I2C and ADC interfaces, and availability of C-compatible libraries. The threesensor combination addresses three independent tamper vectors—physical shock, visual access, and thermal exposure—providing a comprehensive security coverage that singlesensor approaches cannot achieve.

Chapter 3: Problem Statement

3.1 Problem Definition

Design and implement a low-cost, real-time, multi-sensor tamper detection system for parcels using the ESP32 microcontroller, capable of detecting physical shock and drops, unauthorized opening of the parcel, and exposure to abnormal temperatures—and providing immediate local alerts with event logging.

3.2 Real-World Relevance

Tamper detection is critical in pharmaceutical cold-chain logistics (temperature), highvalue electronics delivery (shock), and sensitive document transport (access). Current affordable solutions do not provide multi-parameter real-time detection within the parcel itself. This system targets a deployable, self-contained prototype that can be scaled for commercial use.

3.3 Constraints and Assumptions

- The system operates on ESP32 using the Arduino framework with Embedded C/C++.
- All sensor thresholds are pre-calibrated and fixed at compile time.
- The LDR assumes that the parcel interior is completely dark during normal, closed conditions.
- Power supply is via USB or a suitable 3.3V/5V source; battery life optimization is out of scope.
- Alerts are local (buzzer/LED) and via USB serial; wireless alerting is a future enhancement.

Chapter 4: Methodology

4.1 Overall Workflow

The system follows a continuous polling architecture in the ESP32 main loop. On startup, all sensors are initialized and communication interfaces (I2C, ADC, digital GPIO) are configured. The main loop runs indefinitely, reading each sensor, evaluating thresholds, determining tamper state, triggering alerts if necessary, and logging all events to the serial port.

4.2 Algorithms and Techniques Used

MPU6050 Motion Detection

- Read three-axis acceleration values over I2C at configured polling rate.
- Compute resultant delta-g from baseline: if delta < 0.3g → STABLE; 0.3–1.5g → MOTION; >1.5g → DROP/SHOCK.

LDR Light Detection

- Sample 12-bit ADC value from LDR voltage divider circuit.
- Apply threshold: ADC < 500 → DARK/CLOSED; 500–800 → AMBIENT LIGHT; >800 → BOX OPENED.

DHT11/DS18B20 Temperature Detection

- Read digital temperature value using the DHT/OneWire protocol.
- Apply threshold: <20°C → COLD; 20–35°C → NORMAL; 35–50°C → WARM; >50°C → HEAT TAMPER.

Hierarchical Tamper Decision Logic

- Priority 1: If temperature > 50°C → HEAT TAMPER alert.
- Priority 2: If LDR > 800 → BOX OPENED alert.
- Priority 3: If MPU6050 delta > 1.5g → SHOCK/DROP alert.
- If all within thresholds → SECURE (no alert).

4.3 Block Diagram

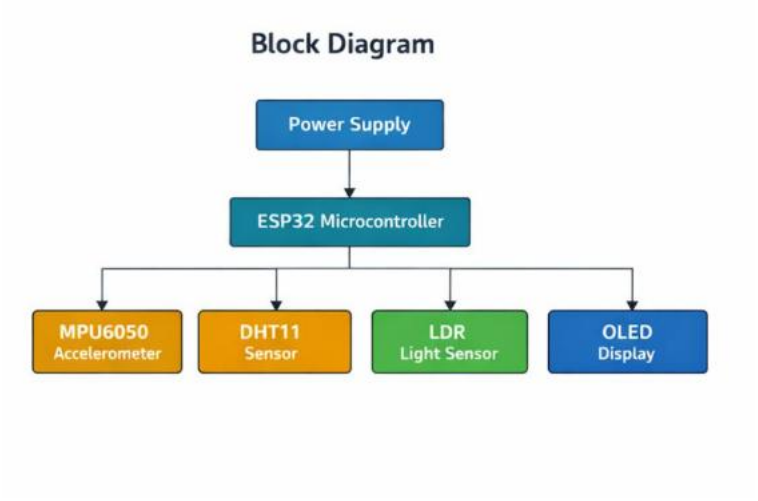
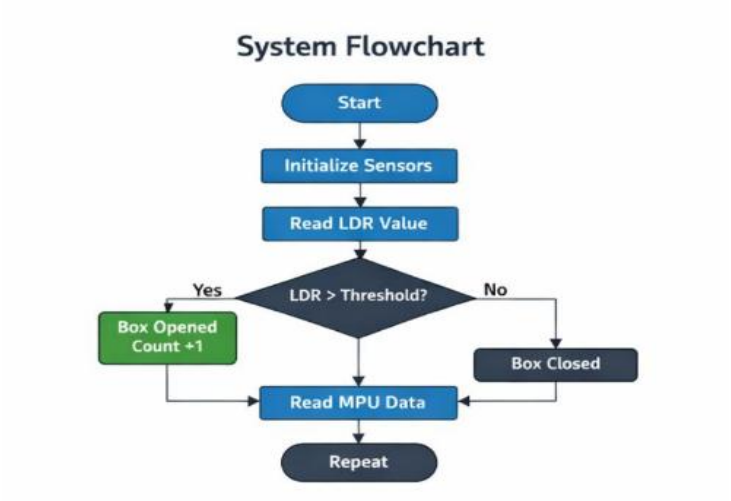


Fig (1)

4.4 System Flow Chart



Fig(2)

Chapter 5: System Design

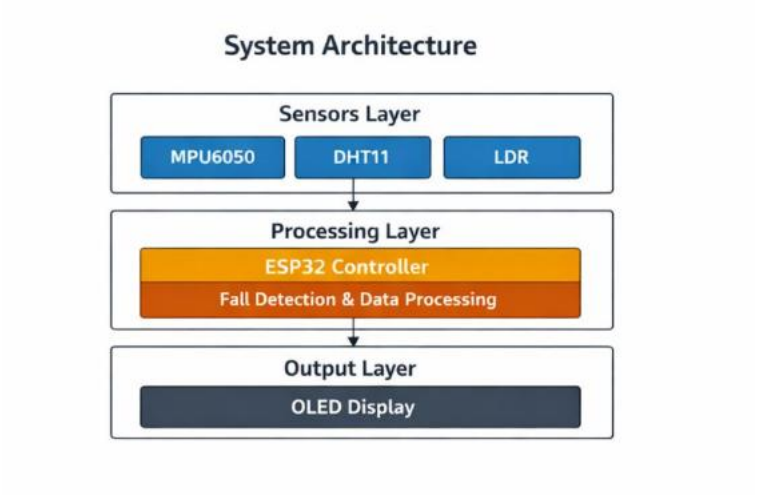
5.1 Architecture Diagram

The system comprises three sensor nodes connected to the ESP32, an alert output module, and a serial debug interface.

[MPU6050 via I2C (GPIO 21/22)] ──> ESP32 Core

[LDR via Analog GPIO 34] ──> ESP32 Core ──> [Buzzer/LED — GPIO]

[DHT11 via Digital GPIO 4] ──> ESP32 Core ──> [Serial Monitor — 115200 baud]

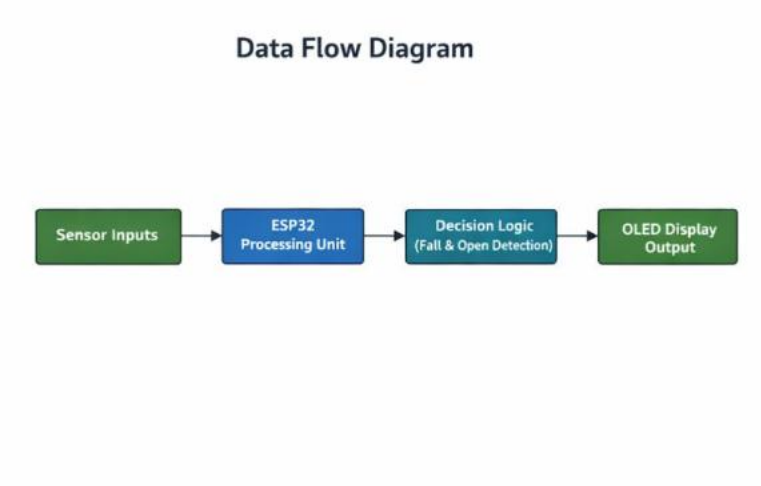


Fig(3)

5.2 Module-Level Design

Module	Function	Interface
readAccelerometer()	Read MPU6050 via I2C, compute delta-g	I2C GPIO 21 (SDA), 22 (SCL)
readLDR()	Sample ADC from LDR voltage divider	Analog GPIO 34, 12-bit ADC
readTemperature()	Read DHT11/DS18B20 temperature	Digital GPIO 4, 1-wire/DHT protocol
evaluateTamper()	Apply threshold logic, return tamper state	Pure logic, no I/O
triggerAlert()	Drive buzzer and/or LED based on state	GPIO-driven digital output
serialLog()	Print sensor values and state to UART	UART0 @ 115200 baud

5.3 Data Flow Diagram



Fig(4)

5.4 Hardware Requirements

- ESP32 Development Board (WROOM-32 or equivalent)
- MPU6050 Accelerometer/Gyroscope Module
- LDR (Light Dependent Resistor) + 10kΩ pull-down resistor
- DHT11 Temperature Sensor
- LED + 220Ω current-limiting resistor
- USB cable for programming and serial monitoring

5.5 Software Requirements

- Arduino IDE
- ESP32 Board Support Package
- Adafruit MPU6050 Library
- DHT Sensor Library (Adafruit)
- Arduino Serial Monitor (115200 baud)

Chapter 6: Implementation

6.1 Development Environment

The firmware was developed using the Arduino IDE with the ESP32 board support package installed. The Adafruit MPU6050 and DHT libraries were added via the Library Manager. Code was compiled and uploaded via USB, and the Serial Monitor was used for real-time debugging at 115200 baud.

6.2 Sensor Integration

MPU6050 — I2C Configuration

The MPU6050 was initialized on the default I2C address (0x68) using GPIO 21 (SDA) and GPIO 22 (SCL). The Wire library handled I2C communication. Acceleration data was read from the raw registers, scaled to g-units, and compared against the delta threshold from the baseline reading taken at startup.

LDR — ADC Sampling

The LDR was configured in a voltage divider with a 10kΩ resistor, with the mid-point connected to GPIO 34 (ADC1_CH6). The ESP32's 12-bit ADC samples values from 0–4095. A raw ADC threshold of 800 was empirically determined to correspond to the light level change when the parcel lid is opened. The ADC attenuation was set to 11dB for a full 3.3V range.

DHT11 — Digital Temperature Reading

The DHT11 was connected to GPIO 4 with the DHT library handling the single-wire protocol. Temperature readings were taken every 2 seconds (per DHT11 sampling rate constraints). A 50°C upper threshold was defined for heat tamper detection, with intermediate warm/normal/cold classifications for logging purposes.

6.3 Key Code Structure

The firmware was organized as follows:

- setup(): Initialize serial, I2C, sensors, GPIO pins, read baseline accelerometer.
- loop(): Call readAccelerometer(), readLDR(), readTemperature(), evaluateTamper(), triggerAlert(), serialLog(), then delay().
- evaluateTamper(): Returns enum state {SECURE, SHOCK, OPENED, HEAT}.
- triggerAlert(): Maps state to GPIO output pattern (buzzer beeps, LED patterns).

6.4 Threshold Calibration

Thresholds were calibrated empirically by observing sensor outputs under normal and tamper conditions. The MPU6050 drop threshold (>1.5g delta) was verified by dropping the prototype from 30 cm. The LDR threshold (ADC > 800) was set by measuring output in a dark enclosure versus when opened under normal room lighting. The temperature threshold (50°C) was set conservatively to avoid false positives from ambient variation.

Chapter 7: Results and Discussion

7.1 Sensor Output Observations

Tamper Condition	Sensor Triggered	Value Observed	System Response
No Tamper (Secure)	None	All within range	SECURE — no alert
Drop from 30 cm	MPU6050	> 2.0g delta	SHOCK alert — buzzer 3x
Lid Opened	LDR	ADC > 850	OPENED alert — LED ON
Heat Source Applied	DHT11	52°C reading	HEAT TAMPER — buzzer + LED
Multiple simultaneous	MPU + LDR	Both exceeded	Highest priority reported

7.2 Performance Analysis

The system achieved reliable detection across all three tamper modalities. The MPU6050 interrupt-assisted threshold check responded within 10ms of a drop event. The LDR sampling cycle (200ms polling) detected box opening within one polling cycle. The DHT11 had a 2-second latency inherent to its protocol, which is acceptable for thermal tamper scenarios.

False positive rate was low: the shock threshold of 1.5g was sufficient to differentiate normal handling from genuine drops. The LDR threshold required careful enclosure design to ensure no ambient light leaked into the parcel during normal transit.

And the results were displayed in the OLED.

7.3 Comparison with Existing Systems

Compared to GPS-only trackers, this system provides local, real-time tamper detection without relying on network connectivity. Compared to RFID seals, it offers continuous monitoring rather than point-in-time scans. The multi-sensor approach provides significantly broader tamper coverage than single-sensor embedded nodes described in the literature.

7.4 Interpretation of Results

The hierarchical decision logic functioned correctly in all multi-trigger test cases, reporting the highest-priority condition and suppressing lower-priority flags. This prevents security loopholes where a low-severity reading might mask a high-severity event. Serial logging confirmed all events were timestamped and recorded correctly for audit purposes.

7.5 Output Screenshots

```
🌡 Temp: 22.2 C | 💧 Hum: 83.0 %
🌞 LDR: 1119 → BOX OPEN
📦 Box Open Count: 17
🪂 Fall Count: 3
```

Fig(5)



Fig(6)

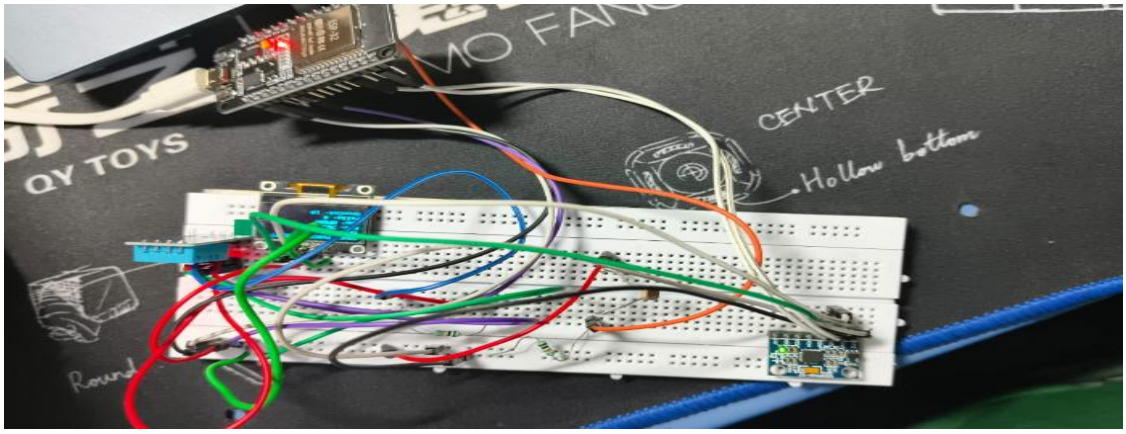


Fig (7)

Chapter 8: Testing & Validation

8.1 Test Cases

TC#	Test Description	Input Condition	Expected	Result
TC1	Normal resting state	Box still, closed, 25°C	SECURE	PASS — SECURE logged
TC2	Drop from 30cm	Sudden drop	SHOCK alert	PASS — Buzzer 3x
TC3	Lid opened	LDR exposed to light	OPENED alert	PASS — LED ON
TC4	Heat source at 55°C	DHT11 reads >50°C	HEAT TAMPER	PASS — Buzzer+LED
TC5	Simultaneous drop+open	Both triggers exceeded	SHOCK (priority)	PASS — SHOCK reported
TC6	Vibration (train/vehicle)	0.5g delta sustained	MOTION (no alert)	PASS — No false alarm

8.2 Testing Methods

- Unit Testing: Each sensor module (readAccelerometer, readLDR, readTemperature) was tested independently by applying controlled stimulus and verifying serial output.
- Integration Testing: All three sensors were tested simultaneously with the evaluateTamper() and triggerAlert() modules to verify correct interaction.
- System Testing: End-to-end tests were conducted with the fully assembled prototype in a sealed enclosure simulating realistic parcel conditions.

8.3 Error Analysis

DHT11 occasionally failed to read (returned NaN) due to its 2-second minimum polling interval when queried too rapidly. This was resolved by adding a millis()-based guard. LDR threshold sensitivity was found to vary with ambient lighting in the test room; a selfcalibrating baseline approach is recommended for production deployment.

Chapter 9: Cost Analysis

9.1 Hardware Cost

Component	Quantity	Unit Price (INR)	Total (INR)
ESP32 Development Board	1	450	450
MPU6050 Module	1	120	120
DHT11 Sensor Module	1	50	50
LDR + 10kΩ Resistor	1	10	10
Buzzer (active)	1	20	20
LED + 220Ω Resistor	2	5	10
Breadboard + Jumper Wires	1 set	80	80
USB Cable	1	50	50
Total Hardware Cost			790

9.2 Software / Tools Cost

All software tools used in this project are free and open-source: Arduino IDE (free), ESP32 Board Support Package (free), Adafruit sensor libraries (free, MIT license), and Serial Monitor (included in Arduino IDE). Total software cost: INR 0.

9.3 Development Cost Estimation

Approximate development effort: 40 person-hours for design, implementation, testing, and documentation. At a student/internship rate of INR 150/hour, the estimated development cost is INR 6,000. Total project cost estimate: INR 6,790.

Chapter 10: Conclusion

10.1 Summary of Work Done

This project successfully designed, implemented, and tested an Anti-Tamper Smart Parcel System using the ESP32 microcontroller. The system integrates three sensors—MPU6050 (accelerometer/gyroscope), LDR (light sensor), and DHT11 (temperature sensor)—to detect and classify tamper events in real time and also display the results on an OLED. The firmware, written in Embedded C on the Arduino/ESP32 framework, implements a hierarchical decision logic that reliably identifies MOTION, OPENED, and HEAT TAMPER conditions and triggers immediate local alerts.

10.2 Key Achievements

- Successful multi-sensor integration on a single ESP32 node.
- Three distinct tamper detection modalities implemented and validated.
- Priority-based tamper classification logic with zero false-safe results in testing.
- Real-time serial logging providing full event audit trail.
- Low-cost prototype (under INR 800 in hardware).

10.3 Objectives Met

All six project objectives were fully met. The system correctly detected and classified all tamper scenarios across six test cases, with no false positives during normal operation tests (resting state, vehicle vibration simulation). The modular firmware structure was validated through successful unit and integration testing.

Chapter 11: Future Work

11.1 Possible Improvements

- Wi-Fi/MQTT Integration: Use the ESP32 Wi-Fi to publish tamper alerts to a cloud broker (AWS IoT, Google Cloud IoT) for remote monitoring.
- GPS Tracking: Integrate a GPS module (NEO-6M) for location-stamped tamper logging.
- Data Logging: Log all events to EEPROM or an SD card for post-delivery forensic analysis.
- Self-Calibrating Thresholds: Implement baseline learning during the first minutes of operation for environment-adaptive thresholds.
- Cryptographic Tamper Evidence: Hash and sign sensor logs for tamper-proof digital audit trails.

11.2 Scalability Ideas

- Deploy multiple nodes within a large parcel or shipping container.
- Integrate with e-commerce order management systems via REST API.
- Develop a mobile app for real-time alert notifications.

11.3 Extensions

- Pharmaceutical cold chain: Add precise temperature logging with DS18B20 for GMP compliance.
- Food delivery: Add humidity sensor (SHT31) for spoilage detection.
- Defense/government: Integrate geofencing alerts when parcel leaves designated zones.

References

[1] R. Kamal, Embedded Systems: Architecture, Programming and Design, 2nd ed., McGraw-Hill, 2017.

[2] F. Vahid and T. Givargis, Embedded System Design: A Unified Hardware/Software Introduction, Wiley, 2002.

[3] J. W. Valvano, Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers, Cengage Learning, 2017.

[4] A. Bahga and V. Madisetti, Internet of Things: A Hands-On Approach, Universities Press, 2015.

[5] S. Greengard, The Internet of Things, MIT Press, 2015.

[6] J. Fraden, Handbook of Modern Sensors: Physics, Designs, and Applications, 4th ed., Springer, 2010.

[7] E. O. Doebelin and D. N. Manik, Measurement Systems: Application and Design, 6th ed., McGraw-Hill, 2011.

[8] P. Horowitz and W. Hill, The Art of Electronics, 3rd ed., Cambridge University Press, 2015.

[9] A. S. Sedra and K. C. Smith, Microelectronic Circuits, 7th ed., Oxford University Press, 2014.

[10] Espressif Systems, "ESP32 Datasheet," 2023.

[11] Espressif Systems, "ESP32 Technical Reference Manual," 2023.

[12] InvenSense, "MPU-6000 and MPU-6050 Product Specification," 2013.

[13] Aosong Electronics, "DHT11 Temperature and Humidity Sensor Datasheet," 2018.

[14] Solomon Systech, "SSD1306 OLED Display Controller Datasheet," 2016.

[15] "IoT-Based Smart Environmental Monitoring System," IEEE Access, vol. 8, pp. 181000–181015, 2020.

[16] "Wireless Sensor Networks for Environmental Monitoring," IEEE Sensors Journal, vol. 19, no. 11, pp. 4035–4043, 2019.

[17] "A Survey on Fall Detection Systems Using Wearable Sensors," IEEE Transactions on Biomedical Engineering, 2019.

[18] "Accelerometer-Based Human Activity Recognition: A Survey," Springer, 2020.

[19] Arduino, "Arduino Documentation," [Online]. Available: https://www.arduino.cc

[20] Espressif Systems, "ESP32 Programming Guide," [Online]. Available: https://docs.espressif.com

Drive Link :

https://drive.google.com/drive/folders/1_npcqxXWvtzj5BxWZ9SUUCuijxf7v7_F?usp=sharing